CRIS: A Hybrid Machine Learning Framework for Phishing URL and Email Detection

Mr. Salman Latheef T A
Assistant Professor
salmanlatheef.ta@rajalakshmi.edu.in.
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Vinothagan K
UG Scholar
230701901@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering.
Rajalakshmi Engineering College
Thandalam, Chennai, India

Janarthanan B
UG Scholar
230701125@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Praveen S
UG Scholar
230701245@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Poovarasan T
UG Scholar
230701228@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Abstract

Phishing remains one of the most enduring and affordable attack vectors in the field of cybersecurity, targeting users with deceptive URLs and fake electronic messages instead of attacking vulnerable software components. A considerable number of prior studies distinguish between phishing URL detection and phishing message detection as two separate challenges and evaluate the algorithms developed separately in carefully crafted single-modality datasets. The paper discusses the detection module of the Cyber Risk Analysis and Intelligence System (CRIS), which represents a hybrid detection architecture based on combining supervised learning and transparent heuristic rule-based approach to analyzing both Uniform Resource Locator (URL) and electronic messages within the same Flask-based web application. Specifically, the CRIS uses a Random Forest classifier for detecting phishing using thirteen structural URL features and risk scoring hand-coded function, while for message detection – a Logistic Regression algorithm with TF-IDF features. The combination of models' results and heuristic reasoning provides a transparent risk conclusion with a corresponding human-readable justification. The paper outlines the detection methodology, system architecture, dashboard implementation, and reporting process and suggests a framework for future dataset-based evaluation of the system. No artificial performance measures are provided; the contribution of the paper is methodological and architectural. The real-time interception of phishing URLs at the moment of user clicking, based on the above detection core, is discussed in a separate work and is out of scope of the paper..

Index Terms

Phishing, Machine Learning, Random Forest, Logistic Regression, TF-IDF, Cyber Security, URLs, E-Mail Security

I. INTRODUCTION

Phishing attacks differ from attacks that exploit software vulnerabilities by fooling the victim into believing they are communicating with a trustworthy entity in order to get credentials, financial information, or other data out of the victim, or to get them to perform some other action on behalf of the attacker. Since the attack exploits the human decision-making process, which is not a vulnerability in the technological infrastructure, the strengthening of the latter does little good to thwart it. In any case, only two methods of delivery prove prevalent in practice - links inserted into webpages, advertisements, or other forms of messaging and messages that prompt their recipient for sensitive data or steer them towards a malicious website - but are researched separately from each other as distinct problems, despite sharing a common root.

A considerable number of contributions have made in the form of machine-learning classifiers for phishing URLs and phishing emails, all of which achieve very high results on publicly available datasets [1]-[10]. Nevertheless, there are three practical drawbacks shared by a majority of these works. Firstly, among the ten papers discussed in this essay, eight focus on detecting phishing websites and URLs, one detects emails for phishing attacks [9] and one detects both spam and phishing [8]; however, none, to the best of our knowledge, combines URL and message detection in one system. Second, the great majority of these studies evaluate a classifier offline on a held-out test partition, as for example in [1], [4], [6], [9], [10], rather than as a component embedded in an operational, user-facing workflow. Third, many systems report a classification label without an accompanying, human-readable justification for that label, which limits the extent to which an end user can reasonably trust or act on the output.

This paper presents CRIS (Cyber Risk Analysis and Intelligence System), a detection platform that combines a Random Forest classifier for structural URL analysis and a TF-IDF/Logistic Regression pipeline for message analysis within a single web application, together with a transparent, hand-coded rule-based scoring layer so that every verdict is accompanied by human-readable justification rather than an opaque probability alone. This detection core is also designed to serve as the foundation for a real-time interception workflow, addressed in a separate study outside the scope of the present paper. The present paper is scoped exclusively to the detection core itself, its methodology, implementation, and evaluation plan.

Section II surveys prior ensemble, deep-learning, and real-time approaches to phishing URL and email detection, and

Section III defines the problem and the project's objectives. Section IV presents the proposed system and its architecture, while Section V details the detection methodology for URLs and messages. Implementation, testing, and evaluation are covered in Sections VI through VIII, and the paper closes with discussion, limitations, future work, contributions, and a conclusion (Sections IX-XIII).

II. Related Work

Most prior work bearing on the CRIS detection core applies ensemble or feature-engineering techniques to URL and website classification. A smaller set of studies targets email or messages, and one survey provides wider context; the literature is therefore organized into the four groups below.

A. Ensemble and Feature-Engineering-Based Phishing URL and Website Detection

Several works in this space build ensembles directly on top of lexical or behavioral URL features. In one line of work, bagging and stacking are applied across multiple Long Short-Term Memory (LSTM) models for phishing URL classification, with mutual-information feature selection narrowing the lexical feature set; the stacking variant came out ahead of the other machine-learning baselines tested in that study [1]. Mutual-information-driven feature selection resurfaces in a Random-Forest-based ensemble aimed specifically at real-time phishing-website detection [3] - a design goal that lines up closely with the real-time, point-of-click motivation behind the broader CRIS project. A large-scale study takes a different angle, pairing Random Forest and Support Vector Machine classifiers with behavior-based URL features and testing the approach against roughly 470,000 URLs sourced from Vietnamese phishing-reporting channels [4]. Among seven classification algorithms compared in a real-time anti-phishing system built on natural-language-processing-derived URL features, a Random Forest classifier using only the NLP-based features came out on top, reaching 97.98% accuracy [6]. A hybrid machine-learning system operating purely on URL-derived features is proposed elsewhere [7]. Last but not least, the EnLeM method uses the voting scheme. That means that there are three classifiers: decision tree, random forest, and k-nearest neighbor that make their own vote after the feature selection procedure, and the overall result is supposed to be better and faster than other methods [10]. A pattern worth noting across this group: Random Forest keeps coming out as a strong, computationally cheap base learner for structural or lexical URL features [3], [4], [6], [10], which is a large part of why it anchors the URL-classification component of CRIS (Section V-B).

B. Feature Engineering and Specialized Detection Approaches

Two further studies address more specialized angles. Muhammad et al. examined redirection-chain patterns as a behavioral feature category for phishing detection [5], a feature category directly analogous to the redirect-pattern feature used within the CRIS URL feature set (Table I, feature 12). MLSPD [8] handles spam and phishing detection jointly in one machine-learning framework; within the literature reviewed here, it is the one work besides [9] whose scope reaches message content such as spam, which makes it relevant to CRIS's own combination of URL and message analysis.

C. Phishing Email Detection

HELPHED takes a hybrid ensemble approach to phishing email detection, running two base learners in parallel over a mix of textual and structural features and combining their outputs through soft voting; it was tested on roughly 32,000 legitimate and 3,500 phishing emails [9]. Of the ten papers reviewed for this study, HELPHED is the only one dedicated to email detection; the rest sit on the URL/website side, apart from the joint spam-and-phishing scope of MLSPD [8]. Section II-D returns to this imbalance, and Section XI flags it as a gap worth closing with further literature review.

D. Surveys and Broader Context

Zieni et al. reviewed the existing literature on approaches to detect phishing websites [2]. This review gives wider perspective of the detection approaches, unlike the approaches reviewed in Sections II-A to II-C, which Do not in themselves suggest a detection approach.

E. Positioning of CRIS Relative to Existing Work

The reviewed literature demonstrates that ensemble and feature-engineering-based machine learning, most commonly built around Random Forest, achieves strong classification accuracy for phishing URL and website detection [1], [3], [4], [6], [7], [10], a finding consistent with CRIS's own selection of Random Forest for URL classification. CRIS differs from this literature primarily in scope and integration rather than in classification technique: rather than optimizing URL-classification accuracy in isolation, CRIS combines URL analysis with message and email analysis within a single operational platform, with each pipeline paired with a transparent rule-based scoring layer so that every verdict is accompanied by interpretable justification rather than a bare label. As noted in Section II-C, only HELPHED [9] among the reviewed papers is dedicated to email detection, and no reviewed work combines URL and message analysis within one integrated platform that also justifies each verdict; this combination, together with the rule-based interpretability layer, constitutes the primary point of differentiation for CRIS. CRIS should therefore be understood as complementary to, rather than a replacement for, the classification techniques reported in [1]-[10]; the ensemble and feature-selection methods surveyed in Section II-A in particular represent a direct avenue for improving the URL-classification component of CRIS, as discussed in Section XI.

III. Problem Statement and Objectives

A. Problem Statement

Phishing attacks are delivered to users via two different but loosely connected channels - malicious URLs and fraud electronic messages. Most of the current academic systems used to detect such attacks are developed as offline classifiers and are tested against a single channel. Moreover, the output of these systems is not easily interpretable by a human being. Thus, the problem that we try to solve in this project can be stated as follows:

Is it possible to merge URL- and message-based phishing detection into one system whose decision will be understandable by humans?

B. Objectives

- Create a machine-learning classifier using structural URL properties to detect phishing links.

- Create another machine-learning classifier based on text to identify phishing messages/emails.
- Couple decisions from both classifiers with a clear scoring system based on explicit rules so that all judgments have an easily understandable explanation.
- Cre ate a central interface for the reporting and history functionality.
- Specify a methodical way of evaluation that does not include fake statistics to facilitate further quantitative assessment of the system.
- Ensure that the detection component is independent enough to allow integration into a real-time interception flow at a later stage of development (the process is considered separately)..

IV. Proposed System

CRIS is a centralised detection system implemented using a Flask-based web application that can be accessed via a manual web dashboard. Every URL or message received from the web dashboard undergoes evaluation using the detection algorithms explained in Section V, and the output generated from the evaluation is then recorded and exported.

A. System Architecture

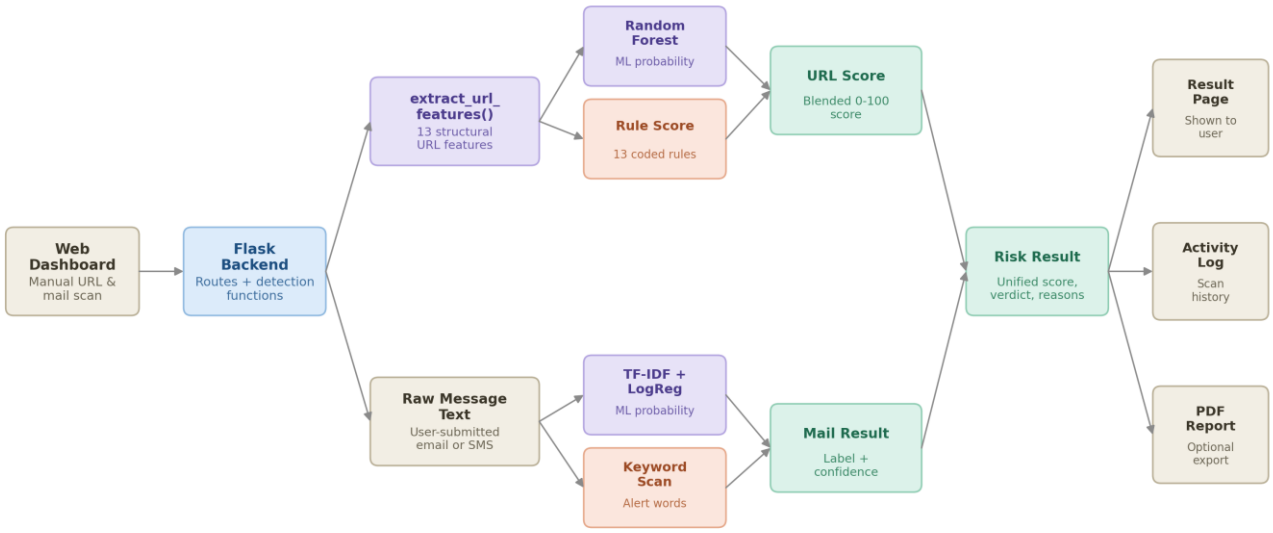


Fig. 1. CRIS detection-core architecture.

B. Manual Analysis Interface

CRIS users will access the tool via a website interface that has two key interfaces; the URL Inspector and the Mail/Message Analyzer. The two tools require a manually inputted URL or message in order to provide its verdict. There are additional screens that make up the interface including the Control Center, which provides quick scan results; Insights, which highlights the frequency of risky keywords; History, which provides previous scan results; and Reports, which provides previous reports.

V. Detection Methodology

URL analysis and message analysis in CRIS are considered independent binary classification tasks that involve both a machine learning model and a rule-based scoring system.

A. URL Feature Engineering

After parsing, all the candidate URLs are represented in a structure feature vector of constant length, shown in Table I below.

TABLE I

URL-Derived Features Supplied to the Classifier

#	Feature	Description
1	URL length	Number of characters in the full URL
2	Dot count	Occurrences of the '.' character
3	Digit count	Occurrences of numeric digits
4	Special chars	Non-alphanumeric symbols present
5	HTTPS	Flag set when the HTTPS scheme is used
6	IP address	Flag set when the host is a raw IP
7	Keyword count	Hits on suspicious terms (e.g., "login")
8	Subdomains	Subdomain levels in the host name
9	'@' symbol	Flag set when '@' appears in the URL
10	Hyphen count	Hyphens appearing in host or path
11	Path length	Characters in the path component
12	Redirect	'//' redirect pattern inside the path

#	Feature	Description
13	Query params	Parameters supplied in the query string

B. URL Classification Algorithm

A feature vector of 13 dimensions is then input to a Random Forest algorithm, which contains 100 decision trees with a maximum depth of 10. Random Forest algorithm was chosen because of the consistency of the performance it has shown as a solid and computationally fast baseline classifier for URL features in previous studies [4], [6], [10].

C. Rule-Based URL Risk Scoring

Independently of the machine-learning classifier, a deterministic rule-based function evaluates the same structural features against a fixed set of heuristics to produce a rule-based risk score in the range 0-100 together with a list of human-readable reasons.

D. Score Fusion and Verdict Classification

The final URL risk score combines the rule-based score with the classifier's predicted phishing probability, expressed as:

$$Score = 0.6 \times R + 0.4 \times (P \times 100) \quad (1)$$

where R is the rule-based risk score and P is the Random Forest's predicted phishing probability. The resulting score is mapped to one of three verdicts - Safe, Suspicious, or Dangerous - using fixed thresholds, and is accompanied by both the triggered rule-based reasons and the per-feature importance derived from the trained Random Forest.

E. Message and Email Feature Representation

The content of messages is encoded through a TF-IDF vectorizer, which uses unigrams and bigrams, removes English stop words, and has a maximum vocabulary size of 3,000, chosen for its lightweight and low-latency nature because of the real-time validation purpose of the whole CRIS project..

F. Message Classification Model

Finally, a Logistic Regression classifier with an inverse regularization value of 1.0 classifies the TF-IDF vector. Like the URL pipeline, the output probability of the classifier is used along with a separate set of heuristic factors such as a list of suspicious keywords, use of too many exclamation marks, and commonly used urgent phrases to give reasons for the classification result..

G. Design Rationale for Model Selection

Selection of Random Forest and Logistic Regression has been made in regard to two project-specific design criteria and not due to any presumption of intrinsic superiority of either method in terms of raw prediction accuracy compared to all other methods discussed in Section II. The first criterion is that both methods are cheap in training and inference in terms of computational costs, which is a requirement because of intended real-time interception workflow, which is a topic of another research, where decision must be made within the budget of one navigation event, while the sequence-based methods, such as the ensemble trained using LSTM in [1], demonstrate the increased training cost and inference overhead introduced by the more complex model. Second, both methods have an easily interpretable internal structure in the form of feature importance in case of Random Forest or coefficient of each term for Logistic Regression with the fixed TF-IDF vocabulary. Such choice allows building rule-based reasoning layer described in Section V-C and V-F without the need for external explainability method. The above selection doesn't prohibit the replacement of the base learner with a stronger model, including the ensemble and feature selection as discussed in Section II-A and II-B [1], [3], [4], [6], [7], [10] on condition that such substitution would preserve the latency and interpretability criteria..

H. End-to-End Processing Pipeline

Regardless of what classifier is used, all submissions go through the same stages of processing: (i) the input data is obtained by the corresponding Flask route; (ii) the input data is converted to its numerical form – a thirteen-dimensional vector for the URL case, or a TF-IDF vector for the message; (iii) the classifier returns the probability of a class; (iv) the independent rule-based module processes the same input and returns the rule score and list of triggering reasons; (v) in the URL case, the probability from the classifier and the rule score are fused using the formula in Section V-D, while in the message case they are just provided side-by-side; (vi) the final decision, confidence scores, and reasons are put into the common result object; and (vii) this result object is simultaneously sent to the requesting web page, added to the activity log, and passed to the PDF report generation module (Section VI-D).

VI. System Implementation

A. Development Approach

First to be developed was the detection engine of CRIS, which defined URL and message classification, scoring based on rules, and the manual dashboard interface, all of which were a stable and independent structure that would later be reused without change by another entry point, the real-time entry point, covered in a different study.

B. Technology Stack

TABLE II

Software Components Used

Component	Technology / Purpose
Server side	Python with the Flask framework
Machine Learning	scikit-learn (Random Forest, Logistic Regression, TF-IDF) and NumPy
Reporting	ReportLab for PDF-based evaluation reports
Frontend	Jinja2 templates with vanilla CSS/JavaScript
Containerization	Docker and Docker Compose
CI / Quality	GitHub Actions running Black, isort, and mypy

C. Application Modules

The features shown in the web dashboard include the following: a Control Center showing the overall level of detection; a URL Inspector and Mail/Message Analyzer to submit manually; an Insights section that provides aggregated information on keywords and risks; a History view that shows past scans; a Reports section to get PDF evaluation reports; and Settings and About pages..

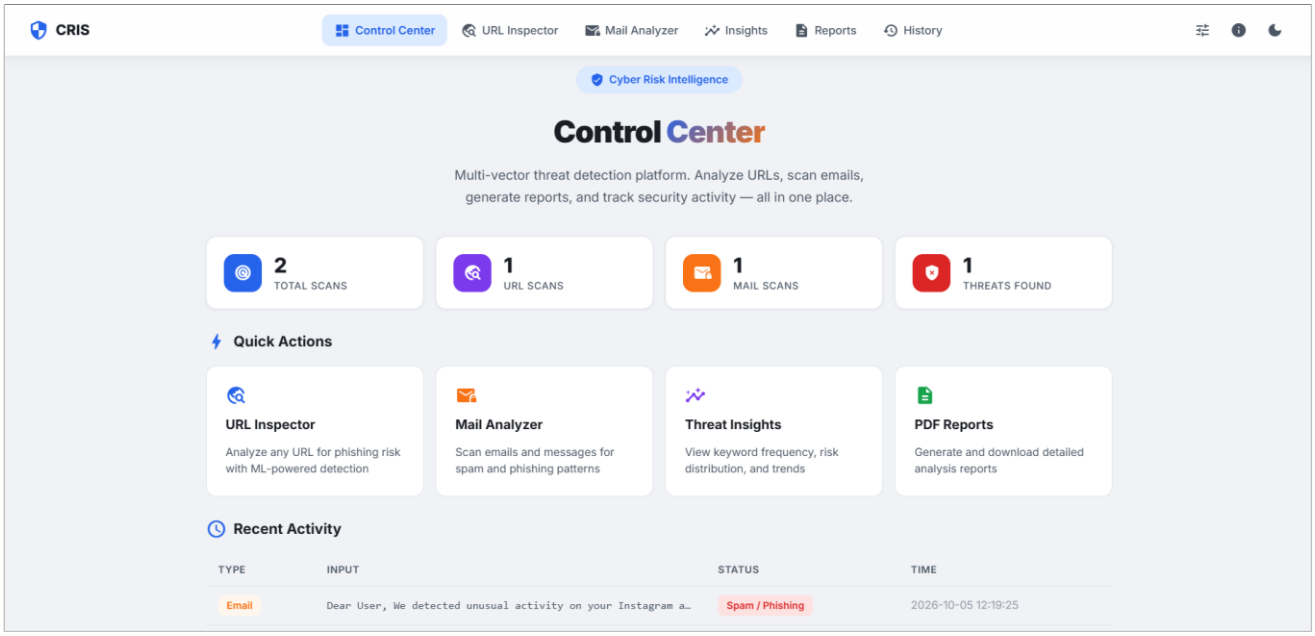


Fig. 2. CRIS Control Center dashboard, showing scan statistics, quick actions, and recent activity.

D. Reporting and Activity Logging

Every evaluation is logged to an activity log that will fill the Control Center view, the History view, and the Insights view and may be optionally exported as a PDF file. The current implementation keeps activities in memory during the lifecycle of the application process but does not store them in a permanent database; this problem is addressed in Section X.

VII. Testing Strategy

Up to now, testing has been done functionally but not formally. All classification and scoring pathways have been put through the system with test URLs and messages, and each run resulted in a full risk analysis - including verdict, confidence scores, and reasons - without any errors.. Unit tests for the individual classifier and feature-extraction functions are not yet in place; they are planned alongside the dataset-driven evaluation described in Section VIII.

VIII. Proposed Evaluation Methodology

No dataset-driven evaluation of the CRIS classifiers has been carried out, which is why this paper quotes no accuracy, precision, recall, or F1 figures. This section instead outlines how such an evaluation should be run.

A. Proposed Dataset Composition

Future evaluation of the URL classifier is intended to use a labeled corpus of real phishing and legitimate URLs, comparable in scale and sourcing to the large, real-world URL collections used in [4] and [6], in place of the synthetic feature distributions currently used for demonstration purposes. Future evaluation of the message classifier is intended to use a real, labeled phishing and legitimate email or message corpus comparable in scale to that used in [9], in place of the small hand-authored example set currently used for demonstration purposes.

B. Proposed Evaluation Protocol and Metrics

The proposed protocol follows conventional supervised-learning evaluation practice: a stratified train/test split (or cross-validation), computation of standard classification metrics on a held-out test partition, and reporting of a confusion matrix for each classifier independently.

TABLE III

Proposed Evaluation Metrics

Metric	Purpose
Accuracy	Fraction of all samples labeled correctly
Precision	Fraction of phishing alerts that were genuine
Recall	Fraction of real phishing cases that were caught
F1-score	Single score combining precision and recall
False-positive rate	Fraction of legitimate items wrongly raised as phishing
False-negative rate	Fraction of phishing cases that slipped through

IX. Discussion

Section II shows that ensembles and engineered-feature classifiers can reach strong accuracy on phishing URL and website data; Sahingoz et al., for instance, report 97.98% accuracy for a Random Forest that uses only natural-language-processing-derived URL features [6]. CRIS does not attempt to exceed such reported figures, and, as stated in Section VIII, does not currently report comparable figures of its own pending a dataset-driven evaluation. The contribution of this work is architectural rather than accuracy-driven: combining URL and message detection within a single platform, with a rule-based interpretability layer that accompanies every machine-learning verdict with human-readable justification - a form of interpretability pursued through a different mechanism than the works reviewed in Section II, the great majority of which report classification performance without an accompanying rule-based justification layer.

Yet another issue to discuss is related to the role of the rule-based scoring layer in relation to the machine learning classifier. In the URL pipeline, the two signals are numerically combined, meaning that if there were a discrepancy between the heuristic rule scores and those

provided by the classifier, then one gets a numerical score anyway instead of getting a discrepancy signal. In the message pipeline, however, the two signals are shown separately, thus providing insight into any discrepancies between the two scores while sacrificing a unified numerical score. This asymmetric approach to combining the two fusion strategies was deliberately preserved for the purpose of the current paper because unification is an empirical question and depends on the behavior of both on real data, as described in Section VIII.

Another aspect that needs discussion is the dashboard-only interaction model used in this project. While it can be seen as a limitation of the detection core, considering the limited usability of a detection mechanism that can be accessed through submitting a URL or a message, this is a conscious decision for the current stage. The fact is that a separate study should be conducted, in which an automatic entry surface would be developed. The reason is that, due to the separation from the entry surface, the functions of the detection core can be reused without any modifications..

X. Limitations

The current implementation carries several limitations that define the scope of the present contribution and distinguish an architectural demonstration from a production-ready, field-validated system.

First, the URL classifier is currently trained on synthetically generated feature distributions rather than on labeled real-world URLs, and the message classifier is trained on a small, hand-authored set of example messages rather than on a real phishing-message corpus. Consequently, no claim is made regarding real-world generalization until the evaluation described in Section VIII is completed on the datasets proposed there.

Second, both classifiers are retrained in memory each time the application starts, rather than being trained offline once and loaded from a persisted model artifact at startup. This does not affect the correctness of a given run, but it is computationally wasteful at scale and means the exact trained model is not guaranteed to be identical between successive runs, which complicates reproducibility of any future reported result.

Third, activity history and generated reports are retained only in memory for the duration of the running application process, rather than in a persistent database; this history is lost whenever the process is restarted, which is acceptable for the current demonstration scope but would not be acceptable for a deployed system expected to retain an audit trail across sessions.

Finally, no quantitative classification results are yet available, as stated in Section VIII, and this paper is scoped exclusively to the detection core itself; a real-time, interactive extension of this system is left entirely to future, separately reported work.

XI. Future Work

Several directions follow from the limitations identified in Section X. The most immediate is replacing the synthetic and hand-authored training data with real-world labeled URL and message datasets and executing the evaluation methodology proposed in Section VIII, since this is the step that would convert the architectural claims made in this paper into measured results.

A second direction is persistence: training the classifiers offline and loading them as saved model artifacts at application startup, together with introducing a persistent database for activity history and reporting data, in place of the current in-memory implementations.

A third direction is evaluating whether the ensemble and feature-selection techniques surveyed in Section II improve classification performance when substituted into the CRIS pipelines, while preserving the latency and interpretability properties discussed in Section V-G.

Finally, the detection core presented in this paper is intended to serve as the foundation for a separately reported extension of the system toward real-time, point-of-click protection; the design and evaluation of that extension are outside the scope of the present paper.

XII. Contributions

Taken together, this work contributes the following.

- An integrated architecture combining independently trained URL and message phishing classifiers with a shared, transparent rule-based scoring layer.
- A score-fusion formulation combining machine-learning probability with rule-based scoring to produce an interpretable, human-readable verdict.
- A centralized dashboard, activity-logging, and PDF-reporting workflow for reviewing detection activity.
- A structured, non-fabricated evaluation methodology intended to guide future quantitative assessment.
- A self-contained detection core designed to serve as the foundation for a later, separately reported extension of the system.

XIII. Conclusion

This paper has described CRIS, which pairs Random-Forest-based URL classification with TF-IDF/Logistic-Regression-based message classification, and ties both to a transparent rule-based scoring layer inside a single Flask-based platform. In keeping with the academic integrity requirements for this work, no performance results have been fabricated; a structured evaluation methodology is proposed in their place. The detection core described here is meant to underpin a real-time interception extension, which is the subject of a separate, future study.

Acknowledgment

The authors thank Rajalakshmi Engineering College, and in particular its Department of Computer Science and Engineering, for the infrastructure and support provided during this work, and their project guide and faculty mentors for ongoing feedback.

References

[1] B. Banik and A. Sarma, "Phishing URL detection using LSTM based ensemble learning approaches," Int. J. Comput. Netw. Commun. (IJCNC), vol. 15, no. 1, pp. 17-33, 2023, doi: 10.5121/ijcnc.2023.15102.

[2] R. Zieni, L. Massari, and M. C. Calzarossa, "Phishing or not phishing? A survey on the detection of phishing websites," IEEE Access, vol. 11, pp. 18499-18519, 2023, doi: 10.1109/ACCESS.2023.3247135.

[3] V. Rani, V. Bardeja, and H. Sharma, "Real-time phishing website detection via mutual information-driven feature selection and

random forest ensemble classification," J. Commun., vol. 21, no. 6, pp. 1-8, 2026, doi: 10.43681/JOCS21.06/2605.

[4] N. T. Lam, "Developing a framework for detecting phishing URLs using machine learning," Int. J. Emerg. Technol. Adv. Eng., vol. 11, no. 11, pp. 61-67, 2021, doi: 10.46338/ijetae1121_08.

[5] T. M. Muhammad et al., "Behavioural feature engineering for phishing detection: The case of redirection patterns," J. Behav. Inform., Digit. Humanit. Develop. Res., vol. 11, no. 2, pp. 106-114, 2025, doi: 10.22624/AIMS/BHI/V11N2P210.

[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Syst. Appl., vol. 117, pp. 345-357, 2019, doi: 10.1016/j.eswa.2018.09.029.

[7] A. Karim, M. Shahroz, K. Mustofa, S. B. Belhaouari, and S. R. K. Joga, "Phishing detection system through hybrid machine learning based on URL," IEEE Access, vol. 11, pp. 36805-36822, 2023, doi: 10.1109/ACCESS.2023.3252366.

[8] S. Kumar, A. Faizan, A. Viinikainen, and T. Hamalainen, "MLSPD - machine learning based spam and phishing detection," in Computational Data and Social Networks (Lecture Notes Comput. Sci., vol. 11280). Springer, 2018, doi: 10.1007/978-3-030-04648-4_43.

[9] P. Bountakas and C. Xenakis, "HELPHED: Hybrid ensemble learning PHishing email detection," J. Netw. Comput. Appl., vol. 210, Art. no. 103545, 2023, doi: 10.1016/j.jnca.2022.103545.

[10] M. N. Yeasmin, Md. A. R. Refat, B. C. Singh, Z. Alom, Z. Aung, and M. A. Azim, "EnLeM: Ensemble learning-based model to detect phishing websites," EURASIP J. Inf. Secur., vol. 2026, Art. no. 1, 2026, doi: 10.1186/s13635-025-00219-1.